

Smart Lab Management System (SLMS)

Software Requirements Specification (SRS) and Complete Project Documentation

Revised Edition — Security & Design Gaps Closed

Final Year Diploma Project Documentation

Version 2.0

Prepared for: Project Mentor / Department of Computer Engineering

Prepared by: Project Team

Academic Year: 2026-2027

Document Control

Item	Details
Project Title	Smart Lab Management System (SLMS)
Project Type	LAN-based desktop/web hybrid monitoring and remote management system
Document Type	SRS + Project Documentation
Version	2.0
Status	Revised working document — supersedes v1.0 after design review
Prepared For	Computer Engineering Department / Project Mentor
Prepared By	Project Team
Core Environment	College computer laboratory connected through LAN

Revision History

Version	Date	Description	Author
1.0	July 2026	Full project documentation prepared from the SLMS master structure	Project Team
2.0	July 2026	Revised after internal design review: added concrete authentication/authorization design, TLS/WSS transport security, client provisioning and identity model, metrics retention strategy, schema migration process, structural RBAC, defined heartbeat/timeout values, API versioning, resolved client OS scope, client resilience design, and expanded test coverage.	Project Team

Summary of Changes from Version 1.0

Version 2.0 closes every gap raised in the independent design review of v1.0. The table below is a quick cross-reference; the corresponding chapters contain the full design detail.

Area	v1.0 gap	v2.0 resolution
Authentication	Required but no mechanism named	JWT bearer-token auth for REST + WebSocket handshake (Ch. 9, 15)
Transport security	Plaintext assumed sufficient on LAN	HTTPS/WSS mandatory, self-signed cert for prototype (Ch. 6, 15, 16)
Client identity	Self-reported hostname/IP/MAC only	Per-agent provisioning credential issued at install time (Ch. 7, 8, 15)
Client privilege	Not discussed	Agent runs as a Windows service; privilege level documented (Ch. 7, 15)
Metrics growth	No retention/archiving plan	30-day raw retention + hourly rollups, indexed (Ch. 8)

Area	v1.0 gap	v2.0 resolution
DB migration	“SQLAlchemy supports it” only	Alembic adopted from day one of backend development (Ch. 6, 13)
RBAC	Prose table only	role_permissions data table + single authorization dependency (Ch. 8)
Status source of truth	REST heartbeat and WebSocket both implied	WebSocket connection state is authoritative; REST is data-only (Ch. 5, 12)
Timing values	“configured interval”, never defined	Defaults: 20s metric interval, 75s offline timeout (Ch. 12)
API versioning	None	All routes under /api/v1/ (Ch. 9)
Client OS scope	Claimed Windows/Linux, designed Windows-only	Prototype scoped explicitly to Windows lab PCs (Ch. 3, 7)
Testing	Single-client happy-path only	Concurrency, reconnection-storm, clock-drift and capacity tests added (Ch. 14)

Abstract

Smart Lab Management System (SLMS) is a client-server software system designed for college computer laboratories. The system helps an administrator monitor all lab computers from a central host computer. Each client computer runs a lightweight Python agent, installed as a Windows service, that collects system information such as CPU usage, RAM usage, disk usage, IP address, operating system details, installed applications, running applications, online/offline status and reported problems. The collected information is sent to the host server through an authenticated REST API and a WebSocket channel secured over the local network.

The administrator views the live status of all computers through a React-based dashboard, identifies problems quickly, maintains issue records, tracks application usage, generates reports and sends approved, whitelisted remote commands such as message, lock, restart or shutdown — every command authenticated, authorized against a role-based permission table, and logged in an audit trail. The project also includes a future-ready AI-assisted module for issue classification, anomaly detection and maintenance recommendation. The system is designed to work inside a college lab LAN without depending on continuous internet connectivity, while still using standard transport encryption and access control rather than relying on network isolation alone.

This revision (v2.0) closes the design gaps identified in an independent review of the original SRS: it defines a concrete authentication and per-agent identity mechanism, mandates TLS/WSS transport, adds a metrics retention and database-migration strategy, converts role permissions into an enforceable data structure, fixes the heartbeat/offline-timeout values, versions the API, resolves the client's operating-system scope to Windows, and expands the test plan to cover concurrency and scale.

The objective of SLMS is to reduce manual lab inspection, improve fault visibility, maintain proper computer records, and support secure remote management with a defensible security design. The system uses Python, FastAPI, Uvicorn, SQLAlchemy, Alembic, SQLite/PostgreSQL, React, Vite, WebSocket (WSS), Tkinter, PyInstaller and Electron in a modular architecture.

Keywords

- Smart Lab Management System
- Computer lab monitoring
- Secure remote management
- FastAPI + JWT authentication
- WebSocket over TLS (WSS)
- Python client agent
- React dashboard
- LAN-based system
- Software inventory
- Maintenance tracking

Table of Contents

1. Introduction
2. Existing System and Literature Survey
3. Problem Statement, Objectives and Scope
4. Requirement Analysis
5. System Architecture
6. Technology Selection and Technology Stack
7. Module Design
8. Database Design
9. API Documentation
10. User Interface Design
11. UML and Behavioral Design
12. Algorithms and Core Logic
13. Implementation Plan
14. Testing and Validation
15. Security, Privacy and Reliability
16. Deployment and User Manual
17. Results and Expected Output
18. Limitations and Future Enhancements
19. Conclusion
20. References
Appendix A: Glossary
Appendix B: Risk Analysis
Appendix C: Project Presentation Points
Appendix D: Security Design Decision Record

Chapter 1: Introduction

1.1 Project Overview

Smart Lab Management System is a software application for monitoring and managing computers in a college laboratory. In a normal lab, teachers or lab assistants must manually check whether each system is working, which applications are installed, which computers have storage or performance issues and which systems require maintenance. SLMS solves this problem by creating a central monitoring system where all client computers report their status to a host server over an authenticated, encrypted channel.

The project follows a host-client architecture. The host computer runs the server, database and administrator dashboard. Each lab computer runs a client agent, installed as a background Windows service and identified to the server by a unique credential issued at installation time — not by self-reported network details alone. The administrator observes all systems from one place, detects issues early and performs permitted remote operations, each of which is authorized against the logged-in user's role and recorded in an audit log.

1.2 Need for the Project

- Manual checking of each lab computer consumes time and effort.
- Faults are often noticed only when a student or teacher reports them.
- There is usually no proper maintenance history for each computer.
- Installed software and running applications are difficult to track manually.
- Administrators need a centralized, access-controlled view of system health, usage and alerts.
- A LAN-based system is suitable because most lab computers are already connected in the same network — but LAN deployment is not treated as a substitute for authentication and encryption.

1.3 Project Vision

The vision of SLMS is to provide a centralized, secure and easy-to-use platform for lab administrators to monitor, manage and maintain computer systems, with security designed in from the first version rather than added later. The system should help the lab move from manual inspection to automated monitoring and planned maintenance.

1.4 Project Users

User Role	Description	Main Responsibilities
Administrator	Main user who controls the system	Monitor systems, view reports, send commands, manage users, roles and settings
Lab Assistant / Technician	Maintenance user	View issues, update maintenance records, fix reported problems
Teacher / Faculty	Observer or limited user	Check lab readiness, view computer availability and issue status
Student	Indirect user	Uses client systems; may submit a structured problem report through the client interface if enabled, attributed to their logged-in lab account

1.5 Client Identification

QR codes are not part of this project. Each client computer is identified primarily by a unique agent credential (a signed token issued when the agent is installed and packaged for that machine). Hostname, IP address, network adapter/MAC information, lab number and system number are stored as descriptive metadata for the administrator's convenience, but they are never used as the sole basis for trusting a client's identity — see Chapter 15 for the full identity and provisioning design.

Chapter 2: Existing System and Literature Survey

2.1 Existing Manual System

In many computer laboratories, lab status is maintained manually. The lab assistant checks systems one by one, records issues in registers and installs or verifies software manually. This approach may work for a small number of systems but becomes inefficient when the number of computers increases.

Existing Practice	Limitation
Manual checking of CPU/RAM/disk status	Slow and not continuous
Manual software verification	Difficult to know which system has missing or unwanted software
Issue register or verbal reporting	Problem history may be incomplete or lost
Physical inspection for every problem	Consumes staff time
No live dashboard	Administrator cannot immediately know system status

2.2 Related Systems

There are many commercial remote monitoring and management (RMM) tools. Such tools typically require paid licenses, internet/cloud accounts or complex setup, but they are also instructive: mature RMM products universally treat device authentication, encrypted transport and signed/whitelisted commands as baseline requirements rather than optional extras. A college project version can adopt that same baseline while keeping scope to local network monitoring, basic remote management, database records, issue tracking and a simple administrator dashboard.

2.3 Learning from Related Concepts

- Network monitoring: Continuous status collection helps identify offline systems and resource problems.
- Asset management: Hardware and software inventory helps the lab maintain proper computer records.
- Remote administration: Controlled remote commands reduce the need for physical access for basic actions — provided the command channel is authenticated and every command is auditable.
- Issue tracking: Problem reports, priorities and maintenance logs improve accountability.
- Analytics and AI assistance: Repeated issues and unusual usage patterns can be converted into useful recommendations.

2.4 Proposed Improvement

The proposed system combines monitoring, inventory, issue reporting, maintenance records, reporting and approved, authenticated remote management into one modular platform designed specifically for a college lab environment, with a security model that would hold up to scrutiny even though the deployment target is a closed LAN.

Chapter 3: Problem Statement, Objectives and Scope

3.1 Problem Statement

College computer laboratories need a centralized system to monitor client computers, track system health, identify software and hardware issues, maintain problem history and perform secure remote management. Manual monitoring is time-consuming and does not provide real-time visibility. Therefore, SLMS is proposed as a LAN-based host-client system for efficient lab monitoring and management, built with authenticated, encrypted communication and role-based access control from the outset.

3.2 Objectives

- To automatically collect computer details from each client system, with each client cryptographically identifiable to the server.
- To show live CPU, RAM, disk and online/offline status in the administrator dashboard.
- To maintain records of installed applications, running applications and usage history.
- To allow users or technicians to report problems and track issue status.
- To generate alerts for low storage, high resource usage or inactive systems.
- To allow authenticated, role-checked, whitelisted remote commands from host to client systems.
- To maintain tamper-evident audit logs for commands and important activities.
- To support future AI-assisted diagnostics and maintenance recommendations.

3.3 Scope of the Project

In Scope	Out of Scope
LAN-based, authenticated and encrypted host-client communication	Public internet-based remote access as a mandatory feature
Computer monitoring and inventory (Windows lab PCs)	Full enterprise device management platform
Issue reporting and maintenance records	Automatic hardware repair
Approved, whitelisted, authenticated remote commands	Unrestricted remote control of every system operation
Reports and analytics	Advanced commercial-level business intelligence
Future-ready AI module	AI auto-executing commands without administrator approval
Per-agent provisioning and identity credentials	QR-code based computer identification
Host redundancy is explicitly out of scope for the prototype (single host, with database backups as the recovery mechanism)	Automatic host failover / multi-server clustering

3.4 Success Criteria

- The host must show connected clients and their latest status, using the WebSocket connection state as the authoritative online/offline signal.

- The client must send system information reliably over an authenticated, encrypted LAN channel.
- The dashboard must show live metrics and issue records clearly.
- The database must store computer, software, usage, issue and command data, with a defined retention policy for high-frequency metrics.
- Remote commands must be permission-checked against a `role_permissions` table, authenticated, and logged.
- The system must continue working inside the lab LAN without mandatory internet, without relying on network isolation as its only security control.

Chapter 4: Requirement Analysis

4.1 Functional Requirements

FR ID	Requirement	Description
FR-01	Client provisioning & registration	Each client computer is issued a unique agent credential at install time and registers with the server using that credential plus hostname, IP, OS and device metadata.
FR-02	System monitoring	Client must collect CPU usage, RAM usage, disk usage, OS details, boot time and network details.
FR-03	Heartbeat & liveness	Client maintains a persistent authenticated WebSocket connection; connection state is the authoritative online/offline signal. Periodic REST calls carry metric payloads only.
FR-04	Software inventory	Client must collect installed application details where supported by the operating system.
FR-05	Running applications	Client must detect active/running applications and filter out unnecessary duplicate system-process noise.
FR-06	Issue reporting	Users or technicians must be able to report problems against a computer using structured fields (category, computer, short description).
FR-07	Maintenance records	Technicians must update repair details, status and remarks.
FR-08	Dashboard	Administrator must view system cards, live metrics, alerts, issues and reports, scoped to what their role is permitted to see.
FR-09	Remote commands	Administrator must send approved commands (message, lock, restart, shutdown) that are authenticated, checked against role_permissions, and delivered only to the verified client identity.
FR-10	Audit log	System must log every authentication event, command, and privileged action with user, computer, timestamp and result.
FR-11	Notifications	System must show alerts for low disk, high resource usage, offline systems or unresolved issues.
FR-12	Reports	System must generate summaries of computer status, issues, usage and maintenance.
FR-13	Authentication & session management	System must authenticate all users and client agents before granting any API or WebSocket access, using signed tokens with a defined expiry.
FR-14	Metrics lifecycle	System must retain raw metric readings for a configurable window (default 30 days) and roll older data up into aggregated summaries.

4.2 Non-Functional Requirements

Category	Requirement
Performance	Client updates should be lightweight (target: under 2% average CPU overhead) and should not affect normal lab usage.

Category	Requirement
Reliability	Server should handle temporary client disconnection and mark systems offline within the defined timeout (Chapter 12).
Security	All API and WebSocket access requires token-based authentication; all traffic runs over TLS/WSS; remote commands require role-based authorization and are logged (Chapter 15).
Usability	Dashboard should be clear enough for teachers and lab assistants to understand quickly.
Maintainability	System should use modular backend, client and frontend structure, with schema changes tracked through Alembic migrations.
Scalability	Design should support at least 40 concurrent clients against the reference host hardware in the prototype, and multiple labs in future.
Portability	Client should be packaged using PyInstaller and installed as a Windows service; host dashboard can be packaged using Electron.
Availability	Core system should work in LAN without mandatory cloud dependency; database backups are the defined recovery mechanism for host failure.

4.3 Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Host computer	Dual-core CPU, 4 GB RAM, 10 GB free disk	i3/i5 or better, 8 GB RAM, SSD
Client computer	Windows 10/11 PC with LAN connectivity	Windows lab PCs with stable wired LAN connection
Network	LAN switch/router	Stable wired LAN for lab systems
Display	Basic monitor for admin dashboard	Large monitor for dashboard view

4.4 Software Requirements

Layer	Software / Tool	Purpose
Client	Python	Client agent development
Client	psutil, platform, socket	System monitoring and identification
Client	Windows API / pywin32	Windows-specific lock/shutdown/app information and running as a Windows service
Client	Tkinter	Small client interface and status display
Client	PyInstaller	Packaging Python client as a Windows service executable
Backend	FastAPI, Uvicorn	REST API, WebSocket server and application server, served behind TLS
Backend	python-jose / PyJWT, passlib	Token issuing/verification and password

Layer	Software / Tool	Purpose
		hashing
Backend	SQLAlchemy	Database access layer
Backend	Alembic	Versioned schema migrations from SQLite to PostgreSQL
Database	SQLite, PostgreSQL	Prototype database and final production database
Frontend	React, Vite	Admin dashboard
Desktop	Electron	Host dashboard desktop packaging
Tools	Git, GitHub, Postman, pgAdmin	Version control, API testing and database management

Chapter 5: System Architecture

5.1 Architecture Style

SLMS follows a modular client-server architecture. The client agent runs on each lab computer as a Windows service. The monitoring server runs on the host computer. The database stores all persistent records. The administrator console displays the data and allows controlled operations. All communication between client and server, and between the dashboard and server, happens through REST API (HTTPS) and WebSocket (WSS) over LAN, protected by token-based authentication.

5.2 Logical Components

Component	Responsibility
Client Agent	Collects system data, maintains an authenticated WebSocket session as its liveness signal, receives approved commands and reports local status.
Monitoring Server	Authenticates every request/connection, validates and stores data, broadcasts live updates, manages the command queue and writes audit logs.
Auth & Identity Service	Issues and verifies user session tokens and per-agent provisioning credentials; central point for all authentication decisions.
Database Server	Stores computers, metrics (with retention/rollup), software, usage history, issues, commands, users, roles/permissions and audit logs.
Administrator Console	Provides dashboard, reports, alerts, issue tracking and command interface, scoped by the logged-in user's role.
AI Assistance Layer	Analyzes repeated problems, abnormal usage and maintenance patterns for recommendations.

5.3 Authentication & Trust Boundaries

Three distinct identities are authenticated by the system, each with its own credential:

- Human users (Administrator, Technician, Teacher) authenticate with a username/password over HTTPS and receive a short-lived JWT session token, used on every subsequent REST call and to open the dashboard WebSocket.
- Client agents authenticate with a long-lived, per-machine provisioning credential generated at installation time (Chapter 15.2) and presented on registration and on every WebSocket reconnect.
- Service-to-database access uses a dedicated database credential read from environment configuration, never from source control (Chapter 15.5).

The WebSocket connection is treated as the trust boundary for liveness: a client is considered online only while its authenticated WebSocket session is active. The periodic REST calls carry data (metrics, inventory) but are not used to infer online/offline state, removing the dual-source-of-truth ambiguity present in the original design.

5.4 Logical Data Flow

1. Client Collection — psutil/platform/socket collect system details.
2. Local Processing — client formats data as JSON and filters duplicate/unwanted process noise.

3. Authenticated Transmission — REST (HTTPS) for periodic data uploads; WebSocket (WSS) for live status and command delivery, both carrying the agent's credential.
4. Server Validation — FastAPI verifies the credential/token, then validates payload format and command permissions against role_permissions.
5. Storage — SQLAlchemy stores status logs, inventory, issues, commands and alerts; a scheduled job applies the metrics retention/rollup policy.
6. Dashboard — Admin views live metrics, issues, software and reports, filtered to their role.
7. Action — Admin sends an approved command or assigns a maintenance task; every action is written to the audit log.

5.5 Deployment View

The host computer runs the Admin Console, Monitoring Server, Auth service and local database during the prototype phase, behind a TLS certificate (self-signed for the LAN prototype, CA-signed if the college later provides one). All lab client PCs connect to the host over the same network switch/router using HTTPS/WSS. Host redundancy is explicitly out of scope for this version (see 3.3); database backups are the defined recovery path if the host fails.

5.6 Communication Methods

Communication Type	Used For	Security
REST API (HTTPS)	Registration, periodic metric/inventory upload, issue creation, reports	TLS transport + bearer token on every request
WebSocket (WSS)	Live metrics, online status, alerts and remote command delivery	TLS transport + token/credential validated at handshake and re-checked per message
Database ORM	Backend to database operations	Dedicated least-privilege DB credential, not embedded in source

Chapter 6: Technology Selection and Technology Stack

6.1 Selected Technology Stack

Part	Selected Technology	Use in Project
Client Language	Python	Build the monitoring agent that runs on every lab computer.
Monitoring Libraries	psutil, platform, socket	Collect CPU, RAM, disk, OS, hostname, IP and network details.
Windows Operations	pywin32 / Windows service API	Run the agent as a Windows service and perform lock/shutdown operations.
Communication	REST (HTTPS) and WebSocket (WSS)	Send periodic data and receive live updates/commands over an encrypted, authenticated channel.
Client Interface	Tkinter	Simple local client window for status and issue reporting.
Client Packaging	PyInstaller	Convert Python client into a Windows-service-installable executable.
Backend Language	Python	Keep backend and client in the same language family.
Backend Framework	FastAPI	Build versioned REST endpoints and WebSocket support with dependency-injected auth.
Server	Uvicorn (behind TLS)	Run the FastAPI application.
Auth	PyJWT / python-jose, passlib[bcrypt]	Issue/verify session tokens; hash user passwords.
Database Access	SQLAlchemy	Define models and interact with SQLite/PostgreSQL.
Migrations	Alembic	Version-controlled schema changes across the SQLite → PostgreSQL lifecycle.
Prototype DB	SQLite	Fast local development and testing.
Final DB	PostgreSQL	Stable multi-user database for final deployment, with table partitioning for metrics.
Frontend	React	Build dynamic administrator dashboard.
Build Tool	Vite	Fast frontend development server and build process.
Host Packaging	Electron	Package admin dashboard as desktop application if needed.

6.2 Why This Stack Was Selected

Technology	Reason for Selection
------------	----------------------

Technology	Reason for Selection
Python	Simple syntax, strong system-monitoring libraries and suitable for student project development.
psutil	Directly supports CPU, memory, disk, process and network statistics.
FastAPI	Modern Python framework with automatic API documentation, validation, WebSocket support and clean dependency injection for auth checks.
PyJWT / passlib	Well-established, widely audited libraries for token handling and password hashing — avoids hand-rolled cryptography.
Uvicorn	ASGI server required to run FastAPI efficiently; runs behind TLS termination.
SQLAlchemy	Reduces raw SQL repetition and provides the model layer Alembic migrates against.
Alembic	Purpose-built schema migration tool; tracks every change as a reviewable, revertible script from the very first schema.
SQLite	No separate database server needed during prototype.
PostgreSQL	Better for the final system with more records, users, reports and native table partitioning for metrics retention.
React	Suitable for building reusable dashboard components and live UI updates.
Vite	Fast React project setup and development experience.
Electron	Allows the host dashboard to behave like a desktop application.
Tkinter	Included with Python, useful for a lightweight client-side interface.
PyInstaller	Helps deploy the client as an installable Windows service rather than a user-run script.

6.3 Alternatives Considered

Alternative	Reason Not Prioritized
Node.js backend	Backend is kept in Python to match the client agent and reduce learning load.
Browser-only dashboard	Browser dashboard is good for testing, but final host may be packaged as a desktop app.
Cloud-only deployment	Not required because the college lab can run fully over LAN, with TLS providing the necessary transport security.
QR-code identification	Removed from project; per-agent provisioning credentials are simpler, more secure and more suitable.
Heavy commercial RMM tools	May require licenses, internet dependency and features beyond project scope.
Network isolation as the only security control	Rejected during design review — a shared lab LAN is not a trusted boundary on its own; authentication and TLS are used regardless of network placement.

Chapter 7: Module Design

7.1 Module List

Module ID	Module Name	Main Responsibility
M-01	Client Monitoring Agent	Collect client computer details and send them to host as a Windows service.
M-02	Client Provisioning & Identification	Issue a unique credential per client at install time and maintain unique, verifiable device records.
M-03	Host Backend API	Receive, authenticate, validate, process and store data.
M-04	Auth & Session Service	Authenticate users and agents, issue/verify tokens, enforce role permissions.
M-05	Real-Time Communication	Provide WebSocket-based live dashboard and command delivery over WSS.
M-06	Admin Dashboard	Show all client computers, alerts, issues, reports and controls, scoped by role.
M-07	Software Inventory	Track installed applications and running applications.
M-08	Usage History	Record application usage and active process information.
M-09	Problem Reporting	Allow structured issue creation, tracking and resolution records.
M-10	Maintenance Management	Store repair history and technician remarks.
M-11	Remote Command Management	Send whitelisted, authorized commands and record command results.
M-12	Alerts and Notifications	Notify high usage, low disk, offline systems and unresolved issues.
M-13	Analytics and Reports	Generate lab status, issue, maintenance and usage reports.
M-14	AI Assistance Module	Provide diagnosis support, anomaly detection and recommendations.
M-15	Security and Audit Log	Track users, permissions, command logs, authentication events and important events.
M-16	Metrics Lifecycle Manager	Apply the retention/rollup policy so raw metrics don't grow unbounded.

7.2 Client Monitoring Agent

The client agent is installed on each lab computer as a Windows service, so it starts automatically at boot, runs independent of any logged-in user, and cannot simply be closed from the taskbar by a student. It collects data at a fixed interval and sends it to the monitoring server over an authenticated channel.

- Collects system details: hostname, IP, OS, CPU, RAM, disk and boot time.
- Collects software inventory and running application details.

- Opens and maintains an authenticated WebSocket session as its online/offline signal.
- Receives command messages through the same authenticated WebSocket channel and executes only whitelisted actions.
- Shows an optional Tkinter status window for issue reporting or connection status, separate from the background service process.

7.3 Client Provisioning & Identification Module

At packaging time, each agent installer is generated with (or securely fetches on first run) a unique provisioning credential tied to that installation. The server treats this credential — not the self-reported hostname, IP or MAC address — as the client's real identity. Hostname/IP/MAC/lab number/system number remain stored as descriptive metadata for the administrator's convenience. Full detail in Chapter 15.2.

7.4 Host Backend Module

The backend is the central brain of SLMS. It authenticates every request, stores data in the database, exposes versioned APIs to the frontend, validates users and handles real-time communication.

- Versioned REST API (/api/v1/...) for registration, metrics upload, issue management and reports.
- WebSocket for live updates and the command channel, authenticated at handshake.
- Database models using SQLAlchemy, migrated with Alembic.
- Business rules for alerts, command permissions (via `role_permissions`) and data validation.

7.5 Admin Dashboard Module

The dashboard is used by authenticated users to understand the complete status of the lab, with screens and actions shown according to the user's role. It provides system cards, filters, charts, issue tables, command buttons and report export options.

7.6 Remote Command Module

Remote commands are restricted to a fixed whitelist, authenticated, authorized against `role_permissions`, and logged. The system never accepts arbitrary operating-system commands from the dashboard — only predefined command types recognized by both server and client. Every command has a status: pending, delivered, executed, failed or cancelled, and one command produces exactly one `command_results` row (see Chapter 8.3 for the corrected cardinality).

7.7 AI Assistance Module

The AI layer is not a first-version requirement. Initially, rule-based diagnostics are used. Later, the system can analyze issue history, repeated failures, high-usage patterns and maintenance records to suggest probable causes and maintenance actions.

7.8 Metrics Lifecycle Manager

A scheduled backend job enforces the metrics retention policy: raw `system_metrics` rows older than the configured window (default 30 days) are aggregated into hourly/daily summary rows and the raw rows are archived or deleted, keeping the primary metrics table bounded in size (Chapter 8.6).

Chapter 8: Database Design

8.1 Database Overview

The database stores all information collected from client computers and all actions performed by users. SQLite is used during development because it is simple and does not require a separate server. PostgreSQL is used for the final system because it is better for multiple users, larger datasets, native table partitioning and stable deployment. Alembic manages every schema change as a version-controlled migration script from the first prototype schema onward, so the SQLite → PostgreSQL move is a data migration, not a redesign.

8.2 Main Tables

Table Name	Purpose
users	Stores administrator, technician and faculty login details and hashed passwords.
roles	Stores role names (Administrator, Technician, Teacher, Client Agent).
role_permissions	Maps each role to the specific actions/endpoints/command types it is allowed to use — the enforceable RBAC structure referenced by the auth dependency.
computers	Stores each lab computer's identity, provisioning credential reference, and basic system information.
client_status	Stores latest online/offline status derived from WebSocket connection state, plus last-seen timestamp.
system_metrics	Stores raw CPU, RAM, disk and network usage records, subject to the retention/rollup policy.
system_metrics_hourly	Stores aggregated (avg/min/max) metric rollups once raw rows age out of the retention window.
hardware_information	Stores processor, memory, disk and OS details.
software_inventory	Stores installed applications and versions.
running_processes	Stores currently running applications/processes.
application_usage	Stores application usage start time, end time and duration.
issue_reports	Stores reported problems and their status, using structured category fields.
maintenance_records	Stores repair actions and technician notes.
remote_commands	Stores commands sent from host to client.
command_results	Stores the single result of each executed command (1:1 with remote_commands).
notifications	Stores alerts and messages shown to users.
audit_logs	Stores authentication events, command activity and important system events.
ai_recommendations	Stores AI/rule-based suggestions for issues and maintenance.

8.3 Important Table Structures

Table	Important Fields
computers	computer_id, lab_no, system_no, hostname, ip_address, mac_address, os_name, os_version, provisioning_credential_id, status, registered_at
role_permissions	permission_id, role_id, action_code (e.g. COMMAND_SHUTDOWN, VIEW_REPORTS), allowed (boolean)
system_metrics	metric_id, computer_id, cpu_usage, ram_usage, disk_usage, network_sent, network_received, recorded_at — indexed on (computer_id, recorded_at)
software_inventory	software_id, computer_id, software_name, version, publisher, installed_date, detected_at
application_usage	usage_id, computer_id, application_name, start_time, end_time, duration_seconds
issue_reports	issue_id, computer_id, reported_by, category, description, priority, status, created_at, resolved_at
remote_commands	command_id, computer_id, command_type, command_payload, issued_by, status, issued_at
command_results	result_id, command_id (unique), executed_at, outcome, detail
audit_logs	log_id, user_id, action, target_type, target_id, ip_address, created_at

8.4 Relationships

- One role can have many role_permissions entries; every authorization check reads this table rather than hard-coded logic.
- One computer can have many system_metrics records, rolled up into system_metrics_hourly once outside the retention window.
- One computer can have many software inventory records.
- One computer can have many issue reports.
- One issue can have many maintenance records or comments.
- One administrator can issue many remote commands.
- Each remote command has exactly one command_results row (corrected from the 1–many ambiguity in v1.0).
- Audit logs are connected to users and important system events, including every authentication attempt.

8.5 Sample SQLAlchemy Model

```
class Computer(Base):
    __tablename__ = "computers"
    id = Column(Integer, primary_key=True, index=True)
    hostname = Column(String, nullable=False)
    ip_address = Column(String, nullable=False)
    os_name = Column(String)
    os_version = Column(String)
    provisioning_credential_id = Column(String, unique=True, nullable=False)
    status = Column(String, default="offline")
    registered_at = Column(DateTime, default=datetime.utcnow)
```

```
class RolePermission(Base):
    __tablename__ = "role_permissions"
    id = Column(Integer, primary_key=True, index=True)
    role_id = Column(Integer, ForeignKey("roles.id"), nullable=False)
    action_code = Column(String, nullable=False)
    allowed = Column(Boolean, default=False)
```

8.6 Metrics Retention Policy

Raw `system_metrics` rows are retained for 30 days by default (configurable in Settings). A scheduled job aggregates rows older than that window into `system_metrics_hourly` (average/min/max per computer per hour) and removes the raw rows, keeping the primary metrics table bounded regardless of how long the system has been deployed. The `(computer_id, recorded_at)` index keeps both the live dashboard queries and the rollup job efficient.

8.7 Schema Migration Process

Alembic is initialized alongside the very first SQLAlchemy models, even while the project still runs on SQLite. Every schema change is captured as a numbered, reviewable migration script. When the project moves to PostgreSQL for final deployment, the same migration chain is replayed against the new database, and existing prototype data is exported/imported table by table — turning the SQLite → PostgreSQL move into a routine data migration rather than a schema reconstruction.

Chapter 9: API Documentation

9.1 API Overview

The backend exposes versioned REST API endpoints under `/api/v1/` for normal request-response operations, and authenticated WebSocket endpoints for live communication. All endpoints require a valid bearer token except the login/agent-registration endpoints themselves, which are rate-limited. All traffic is served over HTTPS/WSS. Every request is validated before storing data or performing actions.

9.2 Authentication Endpoints

Method	Endpoint	Purpose
POST	<code>/api/v1/auth/login</code>	Authenticate a user (username/password) and return a signed JWT session token.
POST	<code>/api/v1/auth/refresh</code>	Exchange a valid refresh token for a new short-lived access token.
POST	<code>/api/v1/agents/provision</code>	One-time endpoint used during agent packaging to register a new provisioning credential for a client machine.

9.3 REST API Endpoints

Method	Endpoint	Purpose
POST	<code>/api/v1/clients/register</code>	Register a client computer using its provisioning credential.
POST	<code>/api/v1/metrics</code>	Upload CPU, RAM, disk and network metrics (authenticated by agent credential).
POST	<code>/api/v1/software/inventory</code>	Upload installed software list.
POST	<code>/api/v1/processes/running</code>	Upload running applications/processes.
GET	<code>/api/v1/computers</code>	Get all registered computers (scoped to caller's role).
GET	<code>/api/v1/computers/{id}</code>	Get details of one computer.
GET	<code>/api/v1/computers/{id}/metrics</code>	Get metric history of one computer (raw + hourly rollups).
POST	<code>/api/v1/issues</code>	Create an issue report.
GET	<code>/api/v1/issues</code>	List issue reports with filters.
PUT	<code>/api/v1/issues/{id}</code>	Update issue status, priority or assignment.
POST	<code>/api/v1/commands</code>	Create an approved remote command; checked against <code>role_permissions</code> before acceptance.
GET	<code>/api/v1/commands/{id}</code>	Check command status and result.
GET	<code>/api/v1/reports/lab-status</code>	Generate lab status summary.
GET	<code>/api/v1/reports/issues</code>	Generate issue report summary.

9.4 WebSocket Endpoints

Endpoint	Connected By	Purpose
/ws/v1/dashboard	Admin dashboard	Receive live computer status, metrics and alerts; connection requires a valid session token at handshake.
/ws/v1/client/{computer_id}	Client agent	Receive remote commands and send live acknowledgements; connection requires the agent's provisioning credential at handshake and is the authoritative online/offline signal for that computer.

9.5 Sample JSON Payload

```
{
  "hostname": "LAB-PC-01",
  "ip_address": "192.168.1.25",
  "os_name": "Windows",
  "cpu_usage": 34.5,
  "ram_usage": 62.1,
  "disk_usage": 78.4,
  "recorded_at": "2026-07-07T10:30:00Z"
}
```

9.6 Sample Authenticated FastAPI Endpoint

```
from fastapi import APIRouter, Depends
from pydantic import BaseModel
from app.auth import get_current_agent, require_permission

router = APIRouter(prefix="/api/v1")

class MetricPayload(BaseModel):
    hostname: str
    ip_address: str
    cpu_usage: float
    ram_usage: float
    disk_usage: float

@router.post("/metrics")
def receive_metrics(payload: MetricPayload, agent=Depends(get_current_agent)):
    # agent identity is verified by get_current_agent before this line runs
    # store payload against agent.computer_id
    return {"message": "Metrics received", "computer": agent.hostname}

@router.post("/commands")
def create_command(cmd: dict, user=Depends(require_permission("ISSUE_COMMAND"))):
    # require_permission checks role_permissions for this user's role
    return {"status": "pending"}
```

9.7 API Validation Rules

- Every REST call and WebSocket connection must present a valid, unexpired token/credential before any other processing occurs.
- Computer identity must be verified via the provisioning credential before accepting metrics or inventory data.
- Only users whose role has the matching role_permissions entry can issue a given command type.
- Command type must be selected from the approved, whitelisted command list.
- Invalid or incomplete JSON should return a clear, structured error response.
- Every authentication event and command request must be recorded in audit_logs.

Chapter 10: User Interface Design

10.1 UI Goals

- Show complete lab status on one dashboard, scoped to the logged-in user's role.
- Use simple cards and tables so non-technical users can understand quickly.
- Show warning indicators for high CPU, high RAM, low disk and offline systems.
- Provide filters by lab, status, issue priority and software.
- Keep remote command buttons clearly separated, permission-gated, and protected by confirmation.

10.2 Main Screens

Screen	Purpose	Main Elements
Login	Authenticate admin/technician against the Auth service	Username, password, role-based redirect after token issue
Dashboard Home	Overall lab view	Total systems, online/offline count, alerts, recent issues
Computer List	View all computers	Computer cards/table, IP, OS, CPU, RAM, disk, last seen
Computer Detail	Deep view of one PC	Hardware info, metrics history (raw + rollups), software, processes, command history
Software Inventory	Track installed software	Software name, version, publisher, installed date, computer list
Issue Management	Track problems	Issue category, priority, status, assigned user, remarks
Maintenance Records	Repair history	Technician, action taken, date, result
Reports	Generate summaries	Lab status, issue report, usage report, maintenance report
Settings	Configure thresholds, roles and timing	CPU/RAM/disk thresholds, heartbeat interval, offline timeout, role permissions, metrics retention window
User & Role Management	Manage accounts and permissions	Create/edit users, assign roles, edit role_permissions matrix

10.3 Client Interface

The client interface is a small Tkinter window showing connection status, hostname, IP address and a structured issue-reporting form (category + short description, rather than open free text). The underlying agent runs as a Windows service independent of this window, so closing or minimizing the window does not stop monitoring or command handling.

10.4 Dashboard Wireframe Description

- Top navigation: project name, user profile, notification icon and logout.

- Summary cards: total computers, online computers, offline computers, active issues and critical alerts.
- Computer grid: each card shows computer name, IP, CPU, RAM, disk and status.
- Right panel: latest alerts, recent commands and unresolved issues.
- Bottom section: reports, usage charts and maintenance summary.

Chapter 11: UML and Behavioral Design

11.1 Use Case Design

Actor	Use Cases
Administrator	Login, view dashboard, view computers, send command (permission-checked), manage users and roles, generate reports, configure thresholds
Technician	Login, view assigned issues, update maintenance record, add repair notes, mark issue resolved
Client Agent	Register with provisioning credential, maintain authenticated WebSocket session, upload metrics, upload software inventory, receive and execute whitelisted command
Teacher / Faculty	Login, view lab readiness, check issue status, report problem if allowed

11.2 Class / Entity Design

Entity / Class	Attributes	Relationships
User	id, name, username, password_hash, role_id	User creates commands, issues and maintenance records
Role	id, name	Role has many role_permissions entries
RolePermission	id, role_id, action_code, allowed	Belongs to one role; consulted on every authorization check
Computer	id, hostname, ip, os, status, lab_no, system_no, provisioning_credential_id	Computer has metrics, software, issues and commands
SystemMetric	id, computer_id, cpu, ram, disk, recorded_at	Metric belongs to one computer
IssueReport	id, computer_id, category, priority, status	Issue belongs to one computer and may have maintenance records
RemoteCommand	id, computer_id, command_type, status, issued_by	Command belongs to one computer, one issuing user, and exactly one CommandResult
Notification	id, type, message, target_user, created_at	Notification may be generated from metrics or issues

11.3 Sequence: Client Status Update

1. Client agent authenticates its WebSocket session using its provisioning credential.
2. Client agent collects CPU, RAM, disk, OS and IP details.
3. Client formats the information as JSON and sends it to the versioned FastAPI metrics endpoint over HTTPS, with its credential attached.
4. Backend verifies the credential and validates the payload.

5. Backend stores latest status and metric history in the database.
6. Backend broadcasts the update to connected dashboard sessions through WSS.
7. Dashboard updates the computer card without a page refresh. The computer's online/offline flag itself is driven by whether its WebSocket session in step 1 is still open, not by this metrics call.

11.4 Sequence: Remote Command

1. Administrator logs in and receives a signed session token.
2. Administrator selects a computer and an approved command.
3. Backend verifies the token, checks role_permissions for that user/command combination, and stores the command as pending.
4. Server sends the command to the connected client's authenticated WebSocket session.
5. Client checks the command type against its local whitelist and executes only the permitted action.
6. Client sends exactly one result back to the server.
7. Server records the result in command_results and writes an audit_logs entry covering the full request → execution → result chain.
8. Dashboard displays the command result.

Chapter 12: Algorithms and Core Logic

12.1 Client Monitoring Algorithm

```

START
Load client configuration and provisioning credential
Collect hostname, IP address, OS and hardware details
Authenticate to server; register if not already registered
Open authenticated WebSocket session (this session = online signal)
LOOP every 20 seconds (default, configurable):
    Collect CPU, RAM and disk usage
    Collect running applications if enabled
    Send metrics to server over authenticated HTTPS
    Check WebSocket for pending command
    If command is valid, whitelisted and approved, execute permitted action
    Send exactly one command result to server
END LOOP
On WebSocket disconnect: attempt reconnect with backoff; server marks
computer offline immediately upon disconnect detection (see 12.2)

```

12.2 Server Heartbeat & Liveness Algorithm

```

START
On WebSocket connect from client:
    Verify provisioning credential
    Set computer status = online
    Broadcast status update to dashboard
On WebSocket disconnect (clean or timeout):
    Set computer status = offline
    Create offline notification
    Broadcast status update to dashboard

On each incoming metrics payload (REST):
    Verify agent credential
    Validate and store metrics
    If metrics cross threshold:
        Create alert notification
    (metrics payloads do NOT change online/offline status)

Every 75 seconds (default offline-timeout, configurable):
    For each computer whose WebSocket has not sent a ping in > 75s:
        Force-close the session and mark offline (safety net for
        silently dropped connections that the transport layer missed)
END

```

12.3 Defined Timing Defaults

Parameter	Default	Configurable in
Metrics/heartbeat upload interval	20 seconds	Settings screen (10.2)
Offline timeout (WebSocket ping tolerance)	75 seconds ($\approx$ 3–4 missed pings)	Settings screen (10.2)
Session token lifetime	15 minutes (access), 8 hours (refresh)	Auth service configuration
Metrics raw-data retention window	30 days before rollup	Settings screen (10.2)

12.4 Alert Generation Rules

Condition	Alert Type	Example Action
CPU usage above threshold for repeated intervals	High CPU alert	Show warning on dashboard
RAM usage above threshold	High RAM alert	Suggest closing heavy applications
Disk usage above threshold	Low storage alert	Mark system for cleanup
WebSocket session closed / offline-timeout reached	Offline alert	Notify lab assistant
Same issue repeated on same computer	Repeated fault alert	Recommend maintenance inspection

12.5 Issue Priority Logic

- Low priority: Minor issue that does not stop system usage.
- Medium priority: Issue affects performance or software availability.
- High priority: Issue prevents normal lab work on one computer.
- Critical priority: Issue affects many systems or complete lab operation.

12.6 AI Assistance Logic

In the first version, AI assistance is implemented as rule-based recommendations. Later, machine learning can be added once enough historical data is collected.

Input Data	Possible Recommendation
Repeated disk full alerts	Clean temporary files or increase storage.
Frequent high RAM alerts	Check startup applications or upgrade memory.
Repeated offline status	Check LAN cable, switch port or network adapter.
Same software missing on many computers	Schedule software installation for the lab.
Frequent issue reports from one computer	Mark computer for detailed maintenance.

Chapter 13: Implementation Plan

13.1 Development Strategy

The project is developed module by module. A small two-computer LAN prototype is built first, with authentication and TLS in place from that very first prototype (not added later), then monitoring, database, dashboard, WebSocket, issue management and remote command modules are added step by step.

13.2 Recommended Folder Structure

```
slms-project/
  backend/
    app/
      main.py
      database.py
      auth.py           # token issue/verify, role_permissions checks
      models/
      schemas/
      routes/
      websocket/
      services/
      metrics_lifecycle.py # retention/rollup job
    alembic/
      versions/
      requirements.txt
  client-agent/
    main.py
    collector.py
    communication.py
    command_handler.py
    ui.py
    service_wrapper.py # Windows service entry point
    config.json        # non-secret connection info only
  admin-dashboard/
    src/
      components/
      pages/
      services/
      App.jsx
    package.json
  docs/
    project-documentation.docx
    api-reference.md
    README.md
```

13.3 Development Phases

Phase	Work	Output
Phase 1	Basic authenticated LAN client-server communication over HTTPS	Client registers with a provisioning credential; server verifies it
Phase 2	System monitoring	CPU, RAM, disk and OS details visible on server
Phase 3	Database integration with Alembic from the start	Client data stored in SQLite, migrations tracked
Phase 4	Admin dashboard with login	React dashboard shows computer list and status behind auth

Phase	Work	Output
Phase 5	WebSocket live updates (authenticated) as the online/offline signal	Dashboard updates without refresh; status reflects connection state
Phase 6	Software and running apps	Inventory and running app information stored
Phase 7	Issue and maintenance module	Structured problem reporting and maintenance history
Phase 8	Remote command module with role_permissions	Approved commands sent to client, authorized, and logged
Phase 9	Reports, alerts and metrics lifecycle job	Lab summaries, threshold alerts, bounded metrics table
Phase 10	Packaging and deployment	PyInstaller Windows-service client and optional Electron host

13.4 Weekly Mentor Reporting Plan

Week	Planned Work	Mentor Review Output
Week 1	Finalize title, problem statement, scope, stack and security design	Approved project abstract, SRS outline and auth/TLS design decision
Week 2	Build basic Python client and FastAPI server with token auth and TLS	Authenticated LAN data transfer demo
Week 3	Add psutil monitoring, database storage and Alembic	System details stored successfully, first migration committed
Week 4	Create React dashboard layout with login	Computer list and status cards behind authentication
Week 5	Add authenticated WebSocket live updates	Real-time CPU/RAM/disk updates; status driven by connection state
Week 6	Add software inventory and running apps	Application monitoring demo
Week 7	Add issue reporting and maintenance records	Issue management workflow
Week 8	Add remote commands with role_permissions and audit logs	Safe, permission-checked command demo and logs
Week 9	Add reports, alerts, metrics rollup job and basic AI rules	Report screenshots and recommendation demo
Week 10	Testing (incl. concurrency/capacity), bug fixing and documentation	Final project report and presentation

13.5 Sample Client Data Collection Code

```
import platform
import socket
import psutil

hostname = socket.gethostname()
```

```
ip_address = socket.gethostbyname(hostname)

system_info = {
    "hostname": hostname,
    "ip_address": ip_address,
    "os_name": platform.system(),
    "os_version": platform.version(),
    "cpu_usage": psutil.cpu_percent(interval=1),
    "ram_usage": psutil.virtual_memory().percent,
    "disk_usage": psutil.disk_usage("/").percent,
}
print(system_info)
```

13.6 Sample Authenticated FastAPI Endpoint

```
from fastapi import FastAPI, Depends
from pydantic import BaseModel
from app.auth import get_current_agent

app = FastAPI()

class MetricPayload(BaseModel):
    hostname: str
    ip_address: str
    cpu_usage: float
    ram_usage: float
    disk_usage: float

@app.post("/api/v1/metrics")
def receive_metrics(payload: MetricPayload, agent=Depends(get_current_agent)):
    # Validate and store payload in database, tied to agent.computer_id
    return {"message": "Metrics received", "computer": payload.hostname}
```

Chapter 14: Testing and Validation

14.1 Testing Strategy

Testing verifies that each module works individually and also works together as a complete system, under both single-client and multi-client conditions. The system is tested on at least two computers connected to the same LAN before expanding to all lab computers, and a simulated multi-client load test is run before capacity claims are made.

14.2 Test Cases

Test ID	Test Scenario	Expected Result
TC-01	Start backend server	Server starts successfully, serves over HTTPS, and API documentation is accessible.
TC-02	Start client agent as a Windows service	Client collects hostname, IP, OS, presents its provisioning credential, and sends a registration request.
TC-03	Unauthenticated request to a protected endpoint	Server rejects with 401 and logs the attempt.
TC-04	Open WebSocket session	Server marks client online and records the session start.
TC-05	Disconnect client network / kill WebSocket	Server marks client offline immediately (or within the 75s safety-net timeout).
TC-06	Upload CPU/RAM/disk metrics	Metrics are stored and visible in dashboard; do not affect online/offline flag independently.
TC-07	Upload installed software list	Software inventory appears for selected computer.
TC-08	Create issue report	Issue is stored with open status and shown in issue table.
TC-09	Update issue status	Issue changes from open to in progress/resolved.
TC-10	Send approved remote command as a permitted role	Client receives command, executes the whitelisted action and returns exactly one result.
TC-11	Send command as a role without permission	Backend rejects the request before it reaches the client and logs the attempt.
TC-12	High disk usage threshold	System generates a low storage alert.
TC-13	Dashboard live update	Metric value changes without page refresh.
TC-14	Report generation	Lab status or issue report is generated correctly.
TC-15	Client packaging as a Windows service	Service installs, starts on boot, and cannot be stopped by a standard user account.
TC-16	Concurrent load: 40 simulated clients	Server remains responsive; no dropped or

Test ID	Test Scenario	Expected Result
	heartbeating and sending metrics simultaneously	corrupted metric rows.
TC-17	Reconnection storm: all clients reconnect at once after a server restart	All clients re-authenticate and re-register without duplicate computer records.
TC-18	Two commands issued to the same computer in quick succession	Both are queued and processed in order; no result cross-contamination between commands.
TC-19	Client/server clock drift	Timestamps recorded server-side (not trusted from client) so ordering of events remains correct.
TC-20	Metrics retention job	Raw rows older than the configured window are correctly rolled up and removed; hourly aggregates match the source data.

14.3 Types of Testing

Testing Type	Purpose
Unit Testing	Check individual functions such as data collection, token verification, permission checks and database insert.
Integration Testing	Check complete flow from client to backend to database to dashboard, including the auth handshake.
System Testing	Verify the full system in an actual lab LAN environment.
Security Testing	Check authentication, TLS configuration, role permissions and remote command restrictions, including negative tests (TC-03, TC-11).
Performance / Load Testing	Check update interval, dashboard response and server load with the target of 40 concurrent clients (TC-16, TC-17).
User Acceptance Testing	Mentor/lab assistant verifies that system solves practical lab problems.

14.4 Validation Checklist

- All connected clients appear in dashboard, identified by their provisioning credential.
- Offline clients are detected correctly via WebSocket disconnect, within the defined timeout.
- Metrics update at the configured interval and do not affect the online/offline flag independently.
- Database records match actual client data.
- Remote commands are authenticated, permission-checked, logged with result status, and never accepted from an unauthorized role.
- Issue reports can be created, updated and resolved.
- Reports contain accurate values and dates.
- The system remains stable under the 40-concurrent-client load test.

Chapter 15: Security, Privacy and Reliability

15.1 Security Principles

- Every REST call and WebSocket connection must present a valid credential/token before any other processing occurs — there is no anonymous access to any data or command endpoint.
- All traffic runs over TLS (HTTPS for REST, WSS for WebSocket); a self-signed certificate is acceptable for the LAN prototype, with a note in deployment (Ch. 16) on moving to a CA-signed certificate if available.
- Passwords are stored using bcrypt (via passlib), never plain text or reversible encryption.
- Remote commands are limited to an approved whitelist and checked against the `role_permissions` table before being queued.
- All command requests and authentication events are logged with user/agent identity, computer, time and result.
- Client-server communication is authenticated regardless of network placement — the LAN is not treated as a trust boundary on its own.
- Input data is validated (via Pydantic schemas) before database storage.

15.2 Client Identity & Provisioning

Hostname, IP address and MAC address are useful descriptive metadata but are not used as the basis for trusting a client, since all three can be changed by anyone with local access to the machine. Instead:

- When the client installer is built for a specific lab deployment (or on first run in a controlled setup step), the server issues a unique, random provisioning credential for that machine and the installer embeds it locally in a protected configuration location.
- Every subsequent registration, metrics upload and WebSocket connection presents this credential; the server looks up the computer record by credential, not by hostname/IP.
- A credential can be revoked from the Admin Console (e.g. if a machine is decommissioned or a credential is suspected to be compromised), immediately cutting off that agent's access without affecting others.

15.3 Client-Side Privilege Model

The client agent runs as a Windows service under a dedicated local service account with the specific privileges needed to lock, restart and shut down the machine — not a full administrator account used for anything else. Running as a service (rather than a per-user foreground process) means:

- The agent starts before any user logs in and keeps running regardless of who is logged in or whether they close any visible window.
- A standard student account cannot stop the service from Task Manager, which preserves the integrity of the “online” signal the dashboard relies on.
- The service account's privileges are scoped to only the operations the whitelist requires, limiting the impact of any bug or compromise in the agent itself.

15.4 Role-Based Access Control

Role	Enforced Via	Typical Permissions
------	--------------	---------------------

Role	Enforced Via	Typical Permissions
Administrator	role_permissions rows for role_id = Administrator	Full dashboard access, user/role management, all command types, reports and settings.
Technician	role_permissions rows for role_id = Technician	View assigned issues, update maintenance records, view computer details; no shutdown/restart commands unless explicitly granted.
Teacher	role_permissions rows for role_id = Teacher	View lab readiness and issue status; limited reporting; no remote commands.
Client Agent	Provisioning credential, not a human role	Send data for its own computer only; execute only commands addressed to it and present on the whitelist.

Because permissions live in a data table rather than scattered code checks, adding a new role or adjusting what a Technician can do is a data change reviewed through the Settings screen, not a code change scattered across route handlers.

15.5 Secrets Management

- Database credentials, the JWT signing key, and the TLS private key are read from environment variables (or a local .env file excluded from version control), never hard-coded or committed to the repository.
- Client config.json contains only non-secret connection information (host address, port); the provisioning credential itself is stored separately with restrictive file permissions, not in the same file as other settings.
- Signing keys are generated per deployment, not reused from a shared example in documentation or source control.

15.6 Remote Command Safety

Remote command handling does not accept free-text operating system commands from the dashboard. It accepts only predefined command types stored in the backend whitelist. The client independently verifies the command type before executing it, and returns exactly one result to the server, which is written to command_results and the audit log.

15.7 Privacy Considerations

- The project collects only technical system data required for lab management.
- Student personal data is not collected; the structured issue-report form (category + short description) intentionally avoids open free-text fields, and reports are attributed to the reporter's logged-in lab account rather than a freely-entered name.
- Screenshots, if implemented in future, would be permission-based and clearly documented before being added to scope.
- Logs are used for maintenance and security review, not unnecessary surveillance.

15.8 Reliability Measures

- WebSocket disconnect is the primary offline signal, with a 75-second ping-timeout safety net for connections that drop without a clean close.
- Retry failed client-server requests after a short delay with exponential backoff.
- Store command status to avoid duplicate execution on retry.
- Use scheduled database backups for the final deployment; host redundancy itself remains out of scope (3.3) with backups as the defined recovery path.
- Show clear error messages when server or network is unavailable.

Chapter 16: Deployment and User Manual

16.1 Prototype Deployment Steps

1. Connect host computer and client computers to the same LAN.
2. Install Python and required packages on development systems; generate a self-signed TLS certificate for the host.
3. Initialize the database and run the current Alembic migration chain.
4. Start the FastAPI backend using Uvicorn (behind TLS) on the host computer.
5. During packaging, generate a provisioning credential for each client machine and embed it in that machine's installer/config.
6. Install the client agent as a Windows service on one client computer and verify authenticated registration.
7. Open the admin dashboard, log in, and confirm that the client appears online (WebSocket session active).
8. Add more clients after the first client-server test succeeds.
9. Use SQLite during prototype and migrate to PostgreSQL for final deployment by replaying the Alembic migration chain and importing data.

16.2 Final Deployment Plan

- Package client agent as a Windows-service-installable executable using PyInstaller.
- Install the client executable on all lab computers, each with its own provisioning credential.
- Configure the client service to auto-start according to lab policy.
- Run the backend server on the host/admin computer or a dedicated server, behind TLS (CA-signed certificate if available).
- Use PostgreSQL for final data storage, with the retention/rollup job enabled.
- Package the administrator dashboard using Electron if desktop application mode is required.
- Create a scheduled backup and restore procedure for the database.

16.3 Administrator User Manual

1. Open the SLMS admin dashboard (served over HTTPS).
2. Login using administrator credentials; a session token is issued and used automatically for subsequent actions.
3. Check dashboard summary cards for total, online and offline systems.
4. Open the computer list to view system metrics and status.
5. Open a computer detail page to view software, usage, issues and command history.
6. Open the issue module to create, update or resolve problem reports.
7. Use remote commands only when required and only for approved lab management purposes; the system will block any command your role is not permitted to issue.
8. Open the reports section to generate lab status, issue and maintenance reports.

9. Use the Settings screen to adjust heartbeat interval, offline timeout, metrics retention window and role permissions as needed.

16.4 Technician User Manual

1. Login with technician account.
2. Open assigned issues.
3. Check issue description, priority and computer details.
4. Perform required maintenance physically or through an approved procedure.
5. Update maintenance record with action taken and result.
6. Mark issue as resolved after verification.

Chapter 17: Results and Expected Output

17.1 Expected Functional Output

Feature	Expected Output
Client registration	Client computer appears in dashboard with hostname and IP address, identified by its provisioning credential.
Live monitoring	CPU, RAM, disk and status values update automatically; status reflects the WebSocket connection state.
Software inventory	Installed applications are listed for each computer.
Running applications	Currently active applications are shown with process details.
Issue report	Problem is stored, assigned and tracked until resolved.
Remote command	Approved, permission-checked command is sent, executed and logged with a single result.
Alert system	Low disk, high usage or offline systems create notifications.
Reports	Administrator can view lab summary, issue summary and maintenance summary.
Security controls	Unauthenticated or unauthorized requests are rejected and logged (verified in TC-03, TC-11).

17.2 Expected Benefits

- Reduces manual computer checking in the lab.
- Improves visibility of lab system health.
- Helps identify problematic systems quickly.
- Maintains proper maintenance history.
- Supports faster decision-making for lab administrators.
- Demonstrates a security design (authentication, encryption, RBAC, audit logging) that would hold up under real scrutiny, not just under a closed-LAN assumption.
- Provides a strong final-year project spanning frontend, backend, database, networking, systems programming and applied security design.

17.3 Screenshots to Add After Implementation

- Login screen screenshot.
- Dashboard home screenshot.
- Computer list and computer detail screenshot.
- Live metric update screenshot.
- Issue report screenshot.
- Remote command history screenshot (showing permission check outcome).
- Report generation screenshot.

Chapter 18: Limitations and Future Enhancements

18.1 Current Limitations

- Prototype capacity target is 40 concurrent clients against the reference host hardware; larger labs would need a capacity re-test.
- Some installed application details may vary depending on operating system permissions.
- Running process list may contain system processes that require filtering.
- AI module requires sufficient historical data before advanced prediction becomes useful.
- Remote command features must follow college policy and Windows service account permissions.
- Host redundancy is out of scope for this version; a host failure requires manual recovery from backup.
- Client scope is Windows lab PCs for this version (Linux support would need an equivalent service/lock/shutdown implementation, not currently designed).

18.2 Future Enhancements

- Multi-lab support with lab-wise dashboard filtering.
- Advanced AI-based predictive maintenance.
- Email or mobile notifications for critical alerts.
- Role-based report export in PDF or Excel format.
- Software blocklist and unauthorized application detection.
- Scheduled maintenance calendar integration.
- Improved client auto-update mechanism, itself authenticated and signed.
- Optional encrypted cloud backup for reports and maintenance history.
- Mobile app for technician issue updates.
- Advanced analytics dashboard with long-term usage trends.
- Host redundancy / warm-standby server for larger deployments.
- Linux client support with an equivalent service/lock/shutdown design.

Chapter 19: Conclusion

Smart Lab Management System is a practical and useful final-year diploma project for managing college computer laboratories. It combines system monitoring, networking, backend development, frontend dashboard design, database management, issue tracking, remote command handling, reporting, applied security engineering and future AI assistance. The project directly addresses real lab problems such as manual inspection, delayed fault detection, missing maintenance history and lack of centralized visibility.

This revised edition closes the design gaps found in the original draft: authentication and authorization are now concrete (JWT sessions for users, provisioning credentials for agents, a `role_permissions` data table enforcing RBAC), transport is encrypted end to end (HTTPS/WSS), client identity no longer depends on spoofable network details, metrics have a defined retention and rollup strategy, schema changes are managed through Alembic from day one, the online/offline signal has a single source of truth, timing values are pinned down, the API is versioned, the client's operating-system scope is resolved to Windows, and the test plan covers concurrency and scale rather than only single-client happy paths.

The selected technology stack remains suitable for the project because Python simplifies client-side monitoring, FastAPI provides a modern backend with clean support for dependency-injected authentication, SQLAlchemy and Alembic together support a safe database evolution path, React supports a dynamic dashboard, and WebSocket over TLS enables real-time communication without sacrificing security. SQLite supports quick prototyping, while PostgreSQL is suitable for final deployment. PyInstaller and Electron help package the client and host parts as desktop applications.

By completing SLMS module by module with security designed in from the start, the project team can demonstrate a complete software engineering workflow from requirement analysis to design, implementation, testing and deployment — one that would hold up to genuine scrutiny, not only to the assumption that a college LAN is inherently safe. The final system can help a college lab administrator monitor all systems more effectively and maintain proper records for future maintenance.

Chapter 20: References

- Python official documentation for core language and standard libraries.
- psutil documentation for system and process monitoring concepts.
- FastAPI documentation for REST API, WebSocket and dependency-injected authentication patterns.
- Uvicorn documentation for ASGI server usage and TLS configuration.
- SQLAlchemy documentation for ORM and database interaction.
- Alembic documentation for schema migration management.
- PyJWT / python-jose and passlib documentation for token handling and password hashing.
- PostgreSQL documentation for relational database concepts and table partitioning.
- React documentation for user interface development.
- Vite documentation for frontend build tooling.
- Electron documentation for desktop application packaging.
- PyInstaller documentation for Python application packaging and Windows service installation.
- OWASP guidance on authentication, session management and API security (general reference for the RBAC and token design).
- Software Engineering and SRS format references used for academic project documentation.

Appendix A: Glossary

Term	Meaning
Client Agent	Software running on each lab computer, as a Windows service, to collect and send data.
Host Computer	Main computer/server used by administrator to monitor clients.
Provisioning Credential	A unique, per-machine secret issued at install time that the server uses to verify a client's identity.
REST API	Request-response communication method between client, backend and frontend, served over HTTPS.
WebSocket (WSS)	Real-time two-way communication channel, encrypted with TLS, used as the online/offline signal.
JWT	JSON Web Token; a signed token used to authenticate a logged-in user's session.
RBAC	Role-Based Access Control; permissions are stored per role in role_permissions and enforced centrally.
Heartbeat	Periodic signal / persistent connection used to show a client is online.
ORM	Object Relational Mapper; used to interact with the database using code models.
Audit Log	Record of important system actions, authentication events and user actions.
Inventory	List of software and hardware details available on a computer.
Rollup	An aggregated summary (e.g. hourly average) created from raw metric rows once they age out of the retention window.

Appendix B: Risk Analysis

Risk	Impact	Mitigation
Client not connecting to server	Dashboard does not show system	Check LAN, host address, TLS certificate trust and server status.
Antivirus blocks client executable	Client may not run	Use proper packaging, clear naming, code-signing if available, and lab administrator approval.
Too many duplicate processes	Running app list becomes unclear	Filter system processes and group duplicate names.
Database grows large	Reports become slow	Metrics retention/rollup job (8.6), PostgreSQL indexing, and archiving.
Unauthorized command attempt	Security issue	Authentication, role_permissions authorization and audit logs (Ch. 15).
Compromised or buggy client agent running with elevated rights	Could affect the lab machine beyond the whitelisted commands	Dedicated least-privilege service account (15.3), whitelist enforced on both server and client.

Risk	Impact	Mitigation
Provisioning credential leaked or stolen	An impersonator could act as that computer	Per-machine revocation from the Admin Console; credentials never logged or displayed after issuance.
Power/network failure	Temporary disconnection	WebSocket reconnect with backoff and the offline-timeout safety net (12.2).
Host machine failure	Full system outage	Scheduled database backups; host redundancy explicitly out of scope for this version (3.3, 18.1).

Appendix C: Project Presentation Points

- Our project solves the problem of manual computer lab monitoring — securely, not just conveniently.
- The system has one host and many client computers connected through LAN, every connection authenticated and encrypted.
- Each client sends CPU, RAM, disk, OS, IP, software and issue details to the host using a credential issued specifically to that machine.
- The admin dashboard shows live status, alerts, software inventory, issues and reports, scoped by the logged-in user's role.
- Remote commands are approved, permission-checked against a role table, and logged for accountability.
- The system works without mandatory internet because it is designed for a local lab network — but it does not rely on the network being trusted; authentication and TLS apply regardless.
- Metrics have a defined retention policy so the database stays bounded over the life of the deployment.
- Future AI module can recommend maintenance actions and detect repeated problems.

Appendix D: Security Design Decision Record

This appendix exists so that, in review or in a viva, the team can point to exactly where and why each security decision was made, rather than treating security as an implicit assumption.

Decision	Chosen Approach	Rationale
User authentication	Username/password + JWT session token (short-lived access, longer-lived refresh)	Standard, well-understood pattern; keeps the server stateless per-request while still allowing logout/expiry.
Agent authentication	Per-machine provisioning credential issued at install time	Hostname/IP/MAC are self-reported and spoofable; a separate secret gives the server something to actually verify.
Transport security	HTTPS + WSS everywhere, self-signed cert acceptable for the LAN prototype	A shared LAN is not inherently private; encrypting regardless avoids relying on network isolation as a security control.
Authorization model	role_permissions data table + single authorization dependency	Centralizes every permission decision in one reviewable place instead of scattered

Decision	Chosen Approach	Rationale
		per-route checks.
Password storage	bcrypt via passlib	Industry-standard, deliberately slow hashing resistant to offline brute force.
Command whitelisting	Fixed, named command types only; no free-text OS commands accepted	Removes the possibility of arbitrary command execution regardless of any other control failing.
Secrets handling	Environment variables / .env, excluded from version control	Prevents the most common student-project mistake of committing credentials to a repository.
Online/offline source of truth	Authenticated WebSocket connection state, with a ping-timeout safety net	Removes the ambiguity of two independent signals (v1.0's REST heartbeat vs. WebSocket) disagreeing with each other.